

sqlc

Por: Ricardo Cuellar



¿Qué es sqlc?

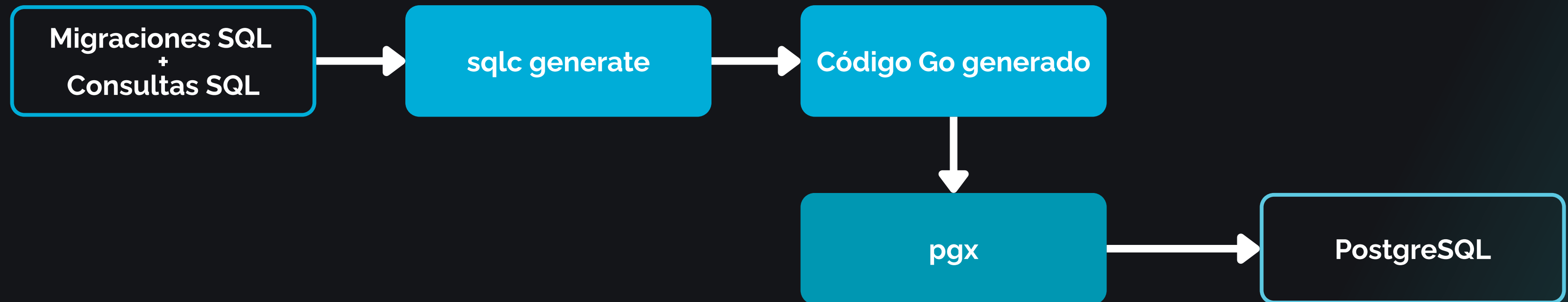
Es un generador de código. Se escribe SQL y sqlc analiza:

- Esquema de base de datos
- Las consultas SQL
- Los parámetros que reciben
- Las columnas y tipos que devuelven

Después genera: funciones, structs e interfaces de Go.



Flujo sqlc



Ejemplo rápido

```
example.sql

CREATE TABLE tasks (
  id UUID PRIMARY KEY,
  project_id UUID NOT NULL,
  title VARCHAR(200) NOT NULL,
  status VARCHAR(20) NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

```
example.sql

-- name: GetTaskByID :one
SELECT
  id,
  project_id,
  title,
  status,
  created_at
FROM tasks
WHERE id = $1;
```



Ejemplo rápido



example.go

```
task, err := queries.GetTaskByID(ctx, id)
```



example.go

```
const getTaskByID = `
SELECT
    id,
    project_id,
    title,
    status,
    created_at
FROM tasks
WHERE id = $1
`

func (q *Queries) GetTaskByID(
    ctx context.Context,
    id uuid.UUID,
) (Task, error) {
    row := q.queryRow(ctx, getTaskByID, id)

    var task Task

    err := row.Scan(
        &task.ID,
        &task.ProjectID,
        &task.Title,
        &task.Status,
        &task.CreatedAt,
    )

    return task, err
}
```



¿Qué es GORM?

GORM es un ORM (Object-Relational Mapper). Intenta interpretar las tablas de la base de datos mediante structs de Go.



Ejemplo con GORM



example

```
SELECT id, project_id, title, status  
FROM tasks  
WHERE id = $1;
```



example.go

```
type Task struct {  
    ID          uuid.UUID  
    ProjectID   uuid.UUID  
    Title       string  
    Status      string  
    CreatedAt   time.Time  
}
```



example.go

```
task, err := gorm.G[Task](db).  
    Where("id = ?", id).  
    First(ctx)
```



GORM ofrece

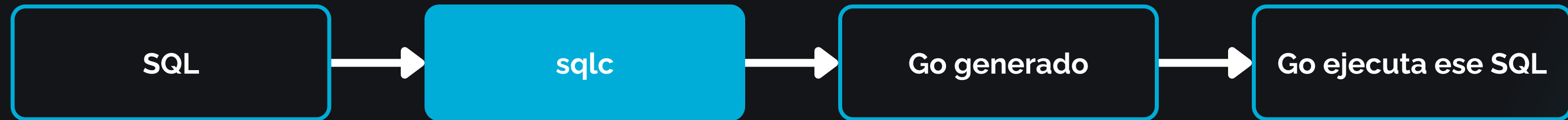
- CRUD
- Relaciones
- Preload y Joins
- Hooks
- Transacciones y savepoints
- Migraciones
- Batch inserts
- SQL Builder
- y más...



En resumen

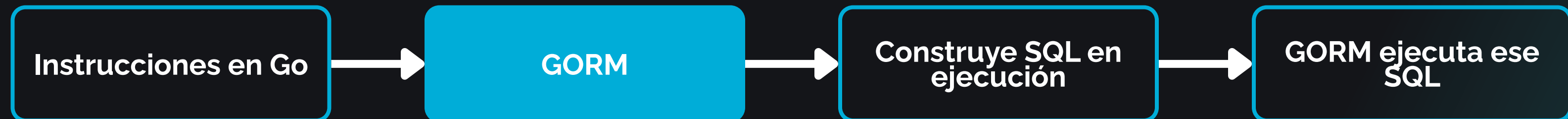
sqlc:

SQL-first



GORM:

Go/model-first



¿Por qué sqlc en lugar de GORM?

- Tienes control completo de que ejecuta cada query
- Errores detectados antes de ejecutar la aplicación
- Los parámetros y respuestas tienen tipos
- Se aprovechan bondades de PostgreSQL
- Menos comportamiento implícito
- Facilita la optimización de consultas
- Refuerzas SQL



Limitaciones de sqlc

- Funciona mejor con consultas definidas y no tan dinámicas
- Conocimientos en SQL (lo básico)
- Genera código
- No maneja migraciones



Gracias



www.devtalles.com



[/ricardocuellars](https://www.linkedin.com/company/ricardocuellars)



[@ricardocuellars](https://twitter.com/ricardocuellars)

